# Group 32 - AskUp Project Documentation

---

**Application Name:** AskUp

**Student Names:** Harshit & Pedro

**Platform:** Android (Kotlin)

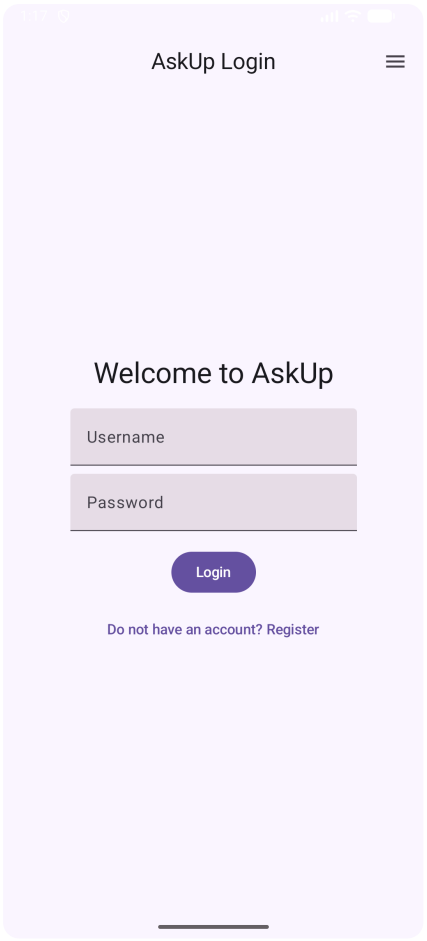
---

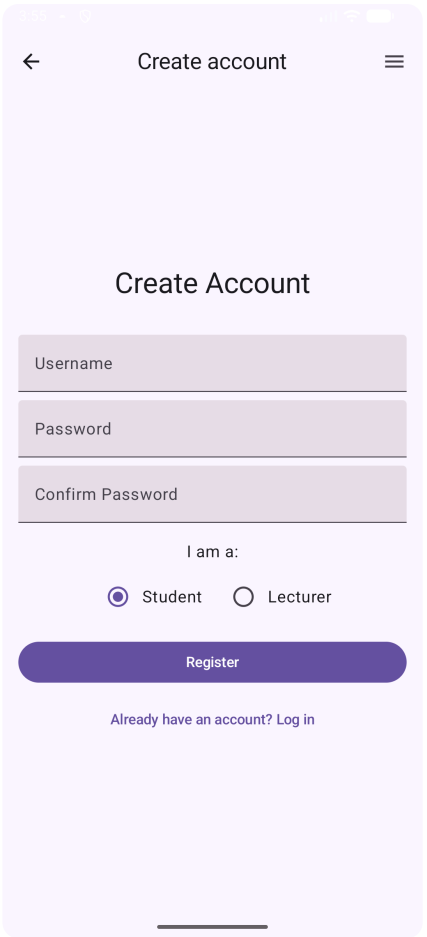
## Introduction

AskUp is an Android application designed to improve classroom engagement by providing students with a digital platform to ask questions during lectures. The application addresses a documented problem where over 70% of university students feel too anxious to ask questions in front of their peers. By offering an anonymous digital alternative, AskUp encourages participation from students who might otherwise remain silent.

The system operates on a session based model where lecturers create classroom sessions using simple 5 digit codes. Students join these sessions by entering the code provided by their lecturer. Once connected to a session, students can post questions, upvote questions from other students, and view answers provided by the lecturer. Lecturers can see all questions from their session, mark them as answered, pin important questions to highlight them, and provide detailed responses.

The technical implementation uses a local Room database for data persistence, ensuring the application works offline and maintains fast response times. All user interface components follow Material Design 3 guidelines to provide a modern and familiar Android experience. The application includes accessibility features such as dark mode support and speech to text input for posting questions.




# User Interface Design

## 2.1 Design Rationale

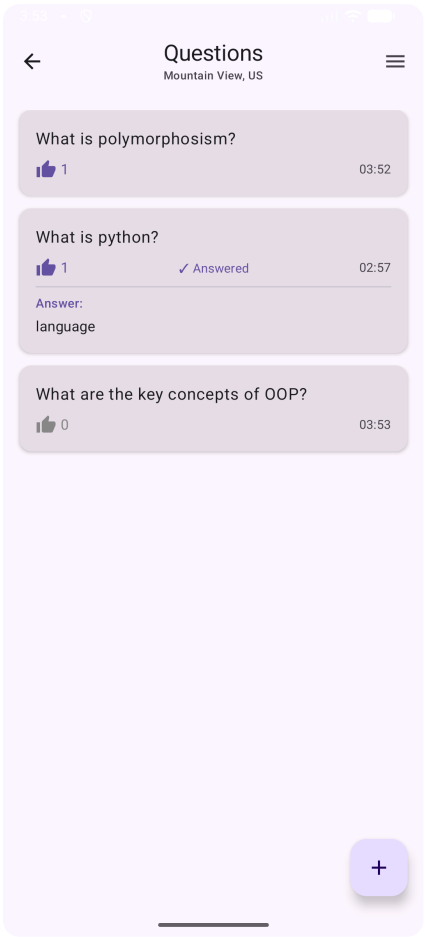
The user interface design for AskUp follows Material Design 3 guidelines to ensure consistency with modern Android applications while maintaining simplicity appropriate for educational contexts. The primary design goal was to create an interface that minimizes cognitive load, allowing users to focus on the core functionality of asking and answering questions rather than navigating complex menu structures.

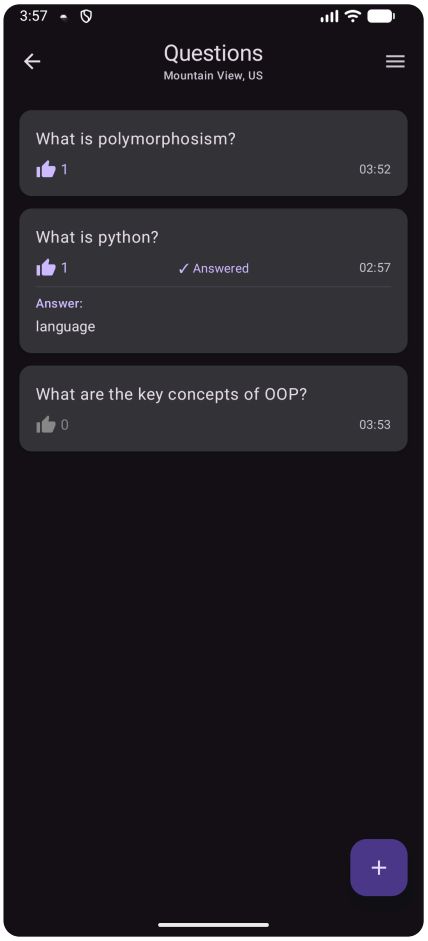
Material Design 3 serves as the foundation for all visual elements. This framework was chosen because it provides consistent interaction patterns that Android users already understand from other applications. The design system includes predefined color schemes for light and dark themes, ensuring proper contrast ratios and readability in different lighting conditions. Using a standard design system also reduces development time since many components come with built in accessibility features.

The application employs a card based layout for displaying questions, which provides clear visual separation between individual items. This design choice supports the assignment requirement for gesture implementation while maintaining intuitive usability. Each question card presents essential information at a glance: the question text, upvote count, answered status, and timestamp. This information hierarchy ensures that users can quickly scan through questions and identify those requiring attention.

Color coding plays a crucial role in conveying state information without requiring text labels. Upvoted questions display a purple thumbs up icon, answered questions show a purple checkmark, and pinned questions feature a distinctive pushpin emoji. These visual indicators work across both light and dark themes, ensuring accessibility for users with different preferences or environmental conditions.

Typography follows a clear hierarchy with three main text sizes. Headlines use the largest size for welcome messages and dialog titles. Body text uses a medium size for question content and general information. Small text appears for metadata like timestamps and upvote counts.



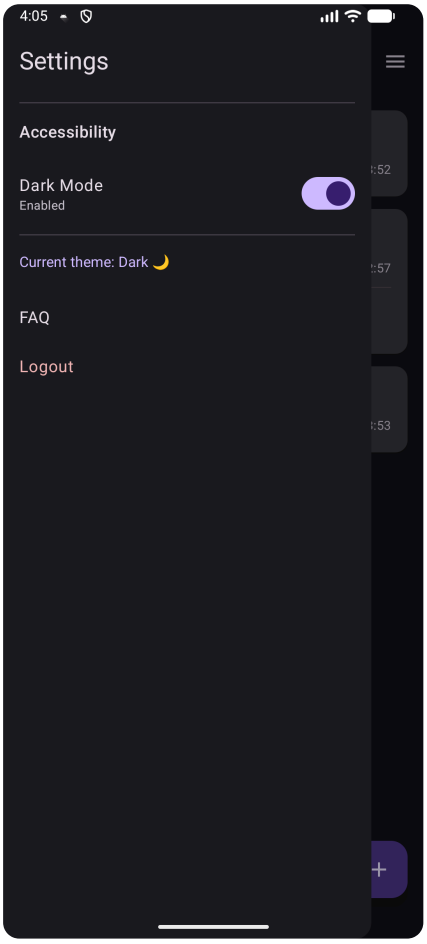




## 2.2 Navigation Architecture

The navigation structure uses a role based layout where students and lecturers follow different paths through the app. After logging in, every user is brought to a `MainActivity` that acts as the central hub and shows the options relevant to their role. This design makes the app easier to understand because users always know they can return to the main screen to reach the key features they need.

Role based navigation ensures students and lecturers see different interface options. When a lecturer logs in, `MainActivity` displays options to create or rejoin a session and proceed to the lecturer view. Students see options to join a session using a code and access the student question view. This separation keeps the interface clean by hiding functionality that isn't relevant to the current user's role.

The application includes a hamburger menu that can be opened from all major screens, giving users consistent access to settings and accessibility options. The menu contains dark mode settings, FAQ information, and logout functionality. Placing these secondary features in a hidden menu keeps the main interface focused on primary tasks while still providing easy access when needed.



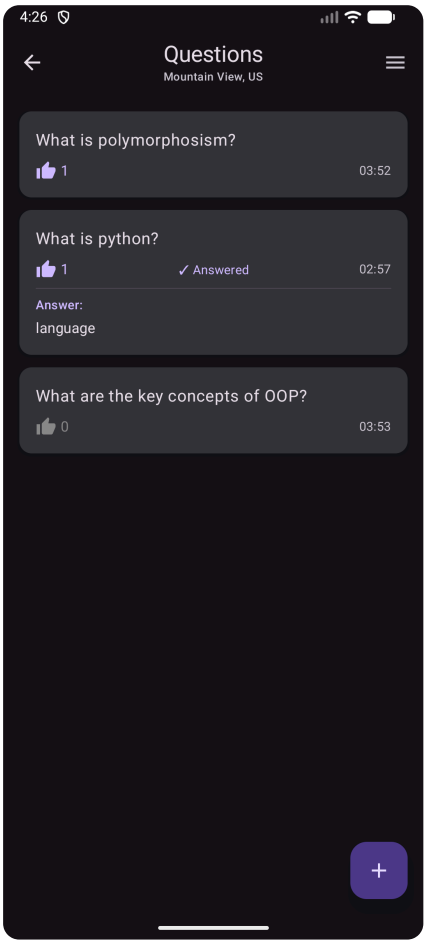

When users press the device back button or tap a back arrow in the top bar, the application returns to the previous screen. The only exception occurs after successful login where pressing back does not return to the login screen. This prevents users from accidentally logging out by pressing back repeatedly.

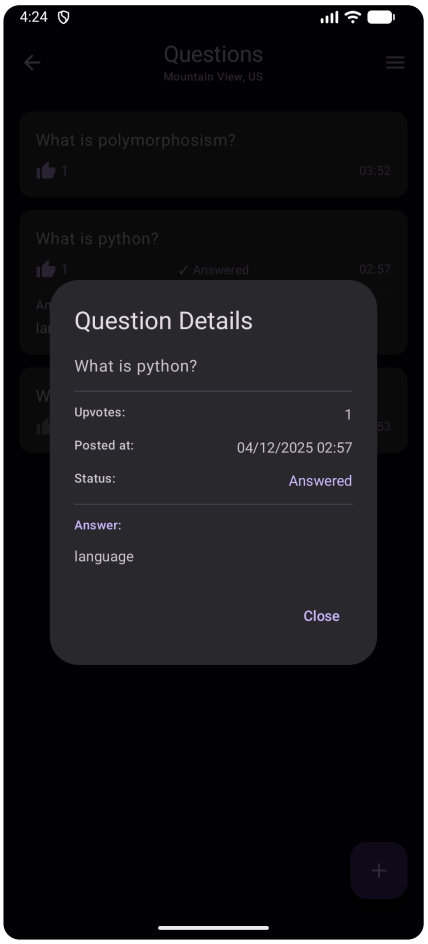
Session management creates a persistent connection between users and their chosen classroom session. Once a student joins a session or a lecturer creates one, the application remembers this choice using `SharedPreferences`. When users return to the application later, they automatically rejoin their previous session without needing to enter the code again.

## 2.3 Widget Layout and Positioning

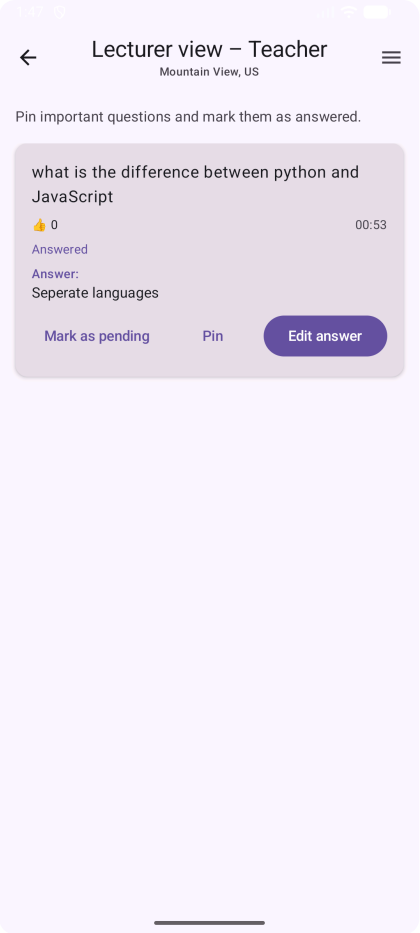
Question cards span the full width of the screen with enough padding to keep the text away from the edges. This makes the content easier to read on mobile devices while keeping the layout comfortable. The cards are arranged vertically in a scrollable list, which matches the natural reading flow and allows the list to grow without needing pagination.

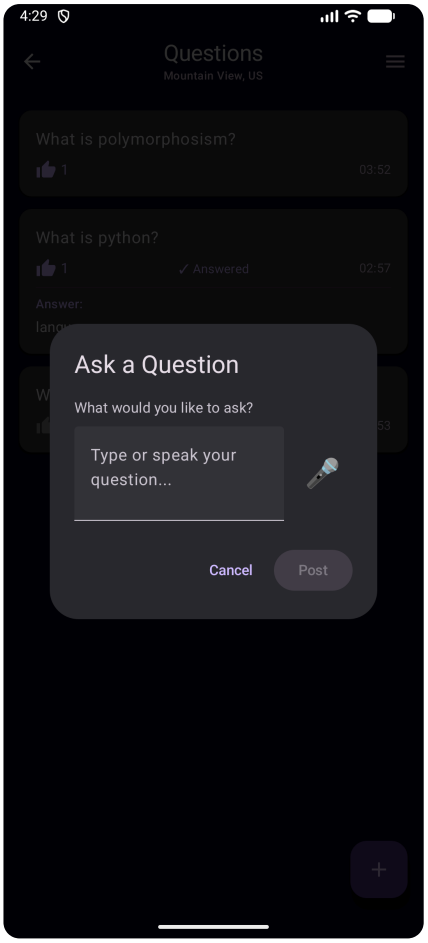
The floating action button for posting questions is placed in the bottom right corner for easy thumb reach on mobile devices. This follows standard Material Design guidelines and keeps the button available at all times, even when scrolling. The plus icon clearly represents adding a new question, so no text label is needed and the interface stays clean.




Upvote counts appear on the left side of each question card next to the thumbs up icon, positioning this interactive element where users naturally begin reading from left to right. The answered status and pinned indicators appear on the right side, creating a balanced visual layout while keeping status information visible without dominating the card. When a question is pinned by the lecturer, it moves to the top of the list, allowing important questions to stand out clearly for all users.




The lecturer interface places action buttons at the bottom of each question card. These buttons are arranged in a horizontal row, with the primary action, adding or editing an answer, positioned on the right and displayed as a filled button. Secondary actions such as marking a question as answered or pinning it use text button styling, giving them less visual weight. This design creates a clear visual hierarchy that directs lecturers toward the most important action while keeping all other options easy to access.

Dialog interfaces follow a centered modal design that dims the background. When users need to enter information such as session codes or question text, a dialog appears in the center of the screen while the rest of the interface darkens to fifty percent opacity. This draws focus to the dialog while still reminding users of their current location in the app. Each dialog includes both confirm and cancel options to allow users to exit an action if needed.

## 2.4 Gesture Interactions

The application implements three distinct gesture types to satisfy assignment requirements while enhancing the user experience. Swipe right gestures trigger upvote actions, long press gestures open detail views, and standard taps handle all button interactions. Each gesture type serves a specific purpose that aligns with user expectations from other mobile applications.

The upvote toggle allows users to remove their upvote by swiping right a second time. This reversible action follows good interaction design principles because it prevents users from being locked into a choice and gives them full control over their input.

# Technical Architecture

## 3.1 Database Design

The application implements a local database using ROOM. This design choice satisfies the assignment requirement for local database usage. The database schema consists of four main entities that represent users, sessions, questions, and voting behavior. Each entity corresponds to a table defined through Kotlin data classes, with Room handling the creation of these tables. This approach ensures a consistent link between the data model in the code and the structure of the database.

## 3.2 Entity Definitions

Foreign key relationships connect entities where logical dependencies exist. Questions reference both the session they belong to and the student who asked them through foreign key columns. The `UserUpvote` entity references both a user and a question, creating a many to many relationship that tracks which users have upvoted which questions

```kotlin
@Entity(tableName = "users")
data class User(
    @PrimaryKey(autoGenerate = true) val userId: Int = 0,
```

The `User` entity stores authentication credentials and role information. Each user record contains a `unique identifier`, `username`, `password`, and role designation as either **student** or **lecturer**. The simple structure reflects the assignment focus on core functionality.

```kotlin
@Entity(tableName = "sessions")
data class Session(
    @PrimaryKey(autoGenerate = true) val sessionId: Int = 0,
```

The Session entity represents a classroom session created by a lecturer. Each session includes a unique session code entered by the lecturer when creating the session. The entity stores a reference to the lecturer who created the session through a foreign key to the users table.

```kotlin
@Entity(tableName = "questions")
data class Question(
    @PrimaryKey(autoGenerate = true) val questionId: Int = 0,
    val sessionId: Int,
```

The Question entity captures all information about a student's query. Foreign keys link each question to the session where it was asked and the student who posed it. The `questionText` field stores the actual question content, which can come from either typed or speech input. The timestamp field automatically records when the question was created using the current system time. The upvotes field maintains a count of how many students have upvoted this question. Boolean flags track whether the question has been answered and whether it has been pinned by the lecturer. The answer field stores the lecturer's response text when provided.

## 3.3 Data Access Objects

The `QuestionDao` provides methods for all question related database operations. The `insertQuestion` method accepts a Question object and returns the auto generated ID of the new row.

Kotlin

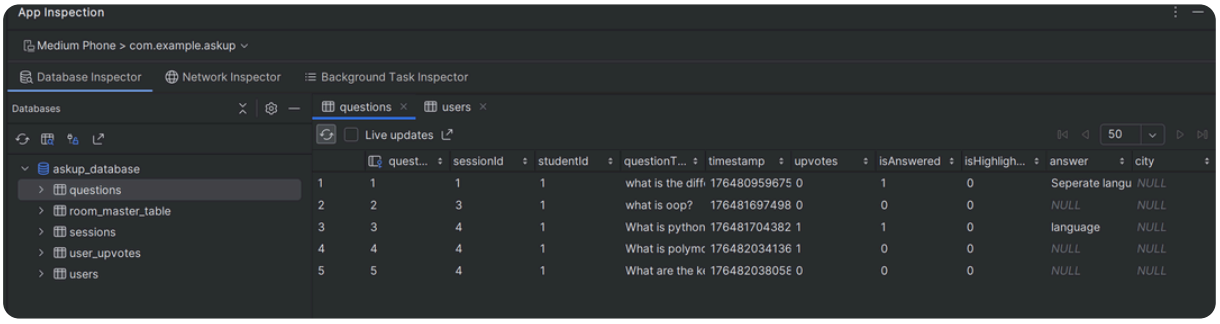
```
@Query("SELECT * FROM questions WHERE sessionId = :sessionId
    ORDER BY isHighlighted DESC, upvotes DESC, timestamp DESC")
fun getQuestionsForSession(sessionId: Int): Flow<List<Question>>
```

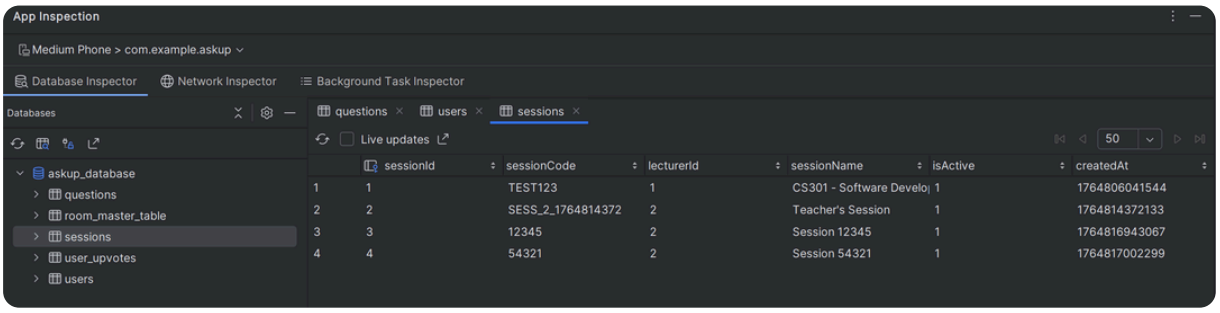
The UserUpvoteDao manages the upvote tracking table. The `hasUserUpvoted` method uses an `EXISTS` subquery to return a boolean indicating whether a specific user has upvoted a specific question. This query executes efficiently because the database can stop searching as soon as it finds a matching row. The `insertUpvote` method adds a new upvote record, while the `removeUpvote` method deletes an existing record based on the composite key of user and question IDs.

The `SessionDao` is responsible for creating sessions and finding them. The getSessionByCode method looks up a session using its code, which students and lecturers use to join a session. This lets the app check that a session code is valid. The `insertSession` method creates a new session when a lecturer enters a unique code.

The `UserDao` handles basic authentication. The `loginUser` method searches the users table for a matching username and password. The `usernameExists` method checks if a username is already in use during registration.

Questions Table




## 3.4 Database State Management

The application uses Kotlin Flow to create reactive data streams from the database. Flow is a coroutine based API that emits values over time, making it ideal for observing database changes. When database content changes, Room automatically triggers a new emission from any Flow that queries the affected table. This mechanism eliminates the need for manual refresh logic throughout the application.

```kotlin
LaunchedEffect(sessionId) {
    database.questionDao().getQuestionsForSession(sessionId)
        .collect { questionList -> questions = questionList }
}
```

`LaunchedEffect` listens to database Flows and updates state variables whenever new data comes in, keeping the UI in sync automatically. It runs inside a coroutine that is tied to the composable's lifecycle, so the work is canceled when the composable leaves the screen.

---

# Core Features Implementation

## 4.1 User Authentication System

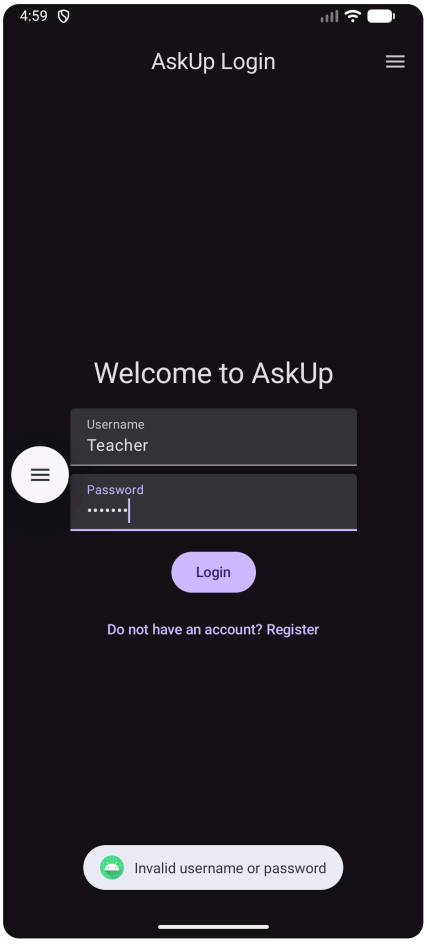
The authentication system validates user credentials against the local database. When users enter their username and password, the application queries the users table using a suspend function that runs on a background thread. The query returns a User object if the credentials match an existing record, or null if no match is found.

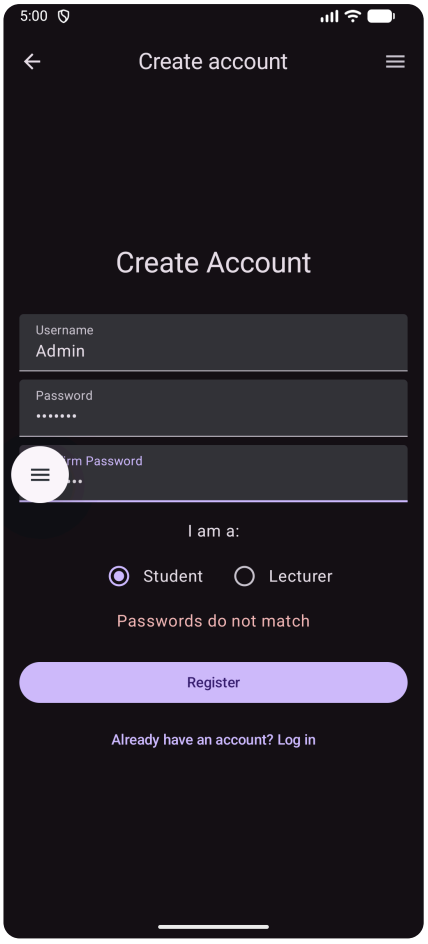
The login process begins when users tap the login button. The button click handler first validates that both username and password fields contain text. If either field is blank, an error message appears without querying the database.

```kotlin
val user = database.userDao().loginUser(username, password)
if (user != null) {
    // Navigate to MainActivity
}
```

Upon successful authentication, the application navigates to `MainActivity` passing the user's ID, username, and role through Intent extras.

The registration process creates new user accounts. Users select their role (student or lecturer) using radio buttons before submitting the form. The registration handler validates that the password meets minimum length requirements and matches the confirmation field. It then checks whether the chosen username already exists in the database using the `usernameExists` query. This check prevents duplicate usernames which could cause confusion during login. If validation passes, the handler creates a new User object and inserts it into the database.




## 4.2 Session Management System

The session system enables multiple independent classrooms to operate within the same application. Each session has a unique 5 digit numeric code that serves as the identifier. Lecturers create sessions by entering a code they choose, while students join sessions by entering the code provided by their lecturer.

When lecturers create a session, the application first validates that no existing session uses the chosen code. This validation query searches the sessions table for any record with a matching `sessionCode`. If the code is already in use, the application displays an error message and prompts the lecturer to choose a different code. If the code is unique, the application creates a new Session object with the entered code, the lecturer's user ID, and a generated session name.

Kotlin

```
val session = Session(
    sessionCode = code,
    lecturerId = userId,
```

The session creation process stores the resulting session ID in `SharedPreferences` along with the code. The next time the lecturer logs in, the application checks `SharedPreferences` for a saved session and automatically loads it, displaying the previously used code on the main screen.

## 4.3 Question Posting and Display

Students post questions using a dialog that appears when tapping the floating action button. The dialog contains a multiline TextField for typing the question text and a microphone button for speech input. The TextField allows multiple lines of text and expands vertically as users type longer questions. A placeholder prompt suggests that users can either type or speak their question.

The speech-to-text feature uses the device's microphone sensor to capture spoken words and convert them into text. When users tap the microphone button, the application starts listening through the microphone and processes the audio from speech recognition. The system uses the device's default language and can display a prompt message before listening begins, then returns the recognized text to the app once processing is complete.

```kotlin
val intent = Intent(RecognizerIntent.ACTION_RECOGNIZE_SPEECH)
intent.putExtra(RecognizerIntent.EXTRA_LANGUAGE_MODEL,
    RecognizerIntent.LANGUAGE_MODEL_FREE_FORM)
```

When speech recognition returns a result, the callback extracts the recognized text from the intent extras. Speech recognition typically returns an array of possible transcriptions.

The question list uses a `LazyColumn` to efficiently show many questions by only composing the items that are currently visible on screen. Each list item is a question card, and the `LazyColumn` reads its data from a state-backed list of Question objects.

The question card composable is responsible for how each question looks. It shows the question text at the top, with a metadata row underneath that includes the upvote count with a thumbs-up icon, the answer status with a checkmark, and a timestamp in short form.

## 4.4 Upvoting System

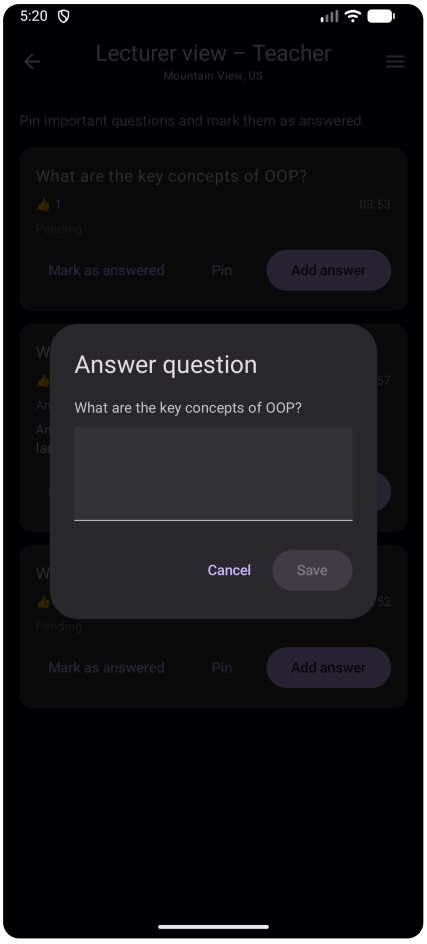
When a student swipes right on a question card, the application first checks whether this user has already upvoted this question. The `hasUserUpvoted` method queries the `UserUpvote` table using an `EXISTS` clause that returns a boolean result. This efficient query stops searching as soon as it finds a match, making it faster than counting rows or retrieving full records.

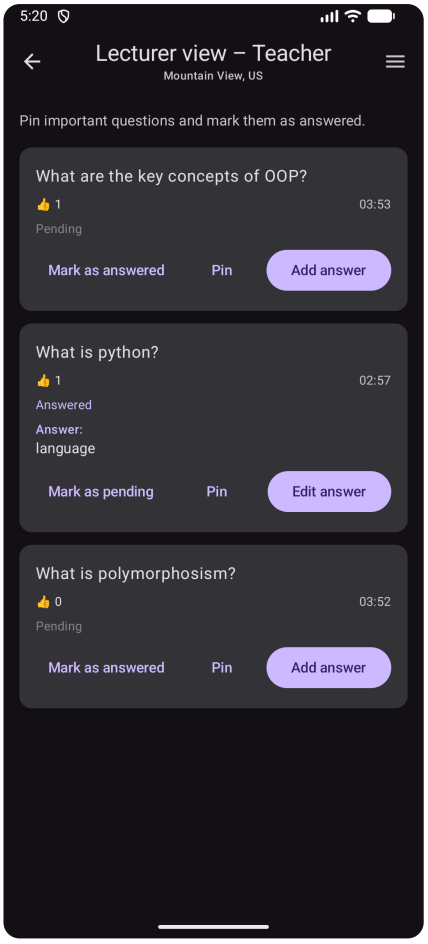
```kotlin
if (alreadyUpvoted) {
    database.userUpvoteDao().removeUpvote(userId, questionId)
```

If the user has already upvoted the question, the swipe removes the upvote. The application deletes the corresponding row from the `UserUpvote` table and decrements the upvote count in the Question record. If the user has not upvoted the question, the swipe adds an upvote by inserting a new row in `UserUpvote` and incrementing the count in Question. Both operations occur within the same coroutine to maintain consistency.

## 4.5 Question Answering by Lecturers

Lecturers answer questions through a dialog interface that shows the original question and provides a text field for composing responses. Each question card in the lecturer view includes an "Add answer" or "Edit answer" button depending on whether an answer already exists. Tapping this button opens the answer dialog.




Kotlin

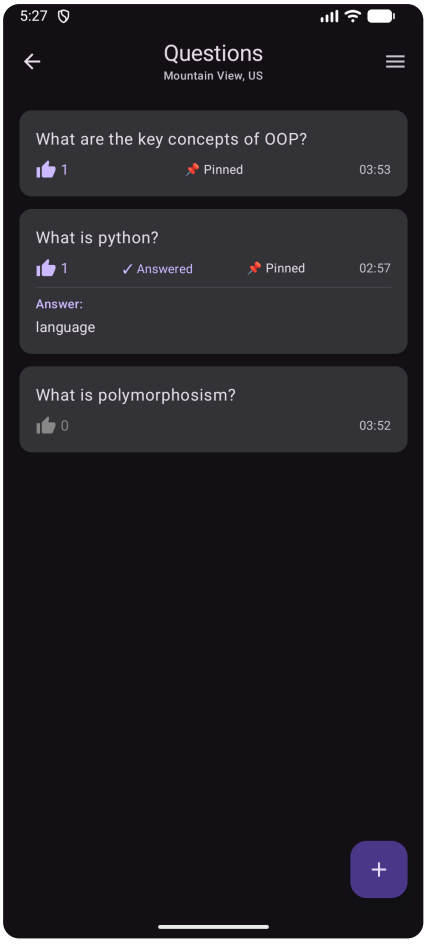
```kotlin
@Query("UPDATE questions SET isAnswered = 1, answer = :answer
    WHERE questionId = :questionId")
suspend fun answerQuestion(questionId: Int, answer: String)
```

The answerQuestion method updates two fields in the question record atomically. It sets the answer text to the provided string and changes the isAnswered flag to true. This single UPDATE statement ensures both changes occur together.

The “mark as answered” toggle lets lecturers change a question’s answered status without editing the answer text. A button labeled “Mark as answered” or “Mark as pending” switches the `isAnswered` flag. This lets lecturers mark questions as handled verbally in class while keeping any existing answer text the same.

## 4.6 Question Pinning Feature

The pin feature allows lecturers to highlight particularly important or frequently asked questions. Pinned questions move to the top of the list regardless of their upvote count, ensuring they remain visible even as new questions are posted. This mechanism helps lecturers maintain focus on key topics during class.




---

# Advanced Features

## 5.1 Speech Recognition Integration

The speech to text functionality provides an alternative input method for posting questions. This feature benefits users who prefer speaking over typing, users with physical limitations that make typing difficult, and situations where users need to ask questions while their hands are occupied.

The recognized text populates the question TextField automatically through the callback function. Users can see the transcribed text and make any necessary corrections before submitting the question.

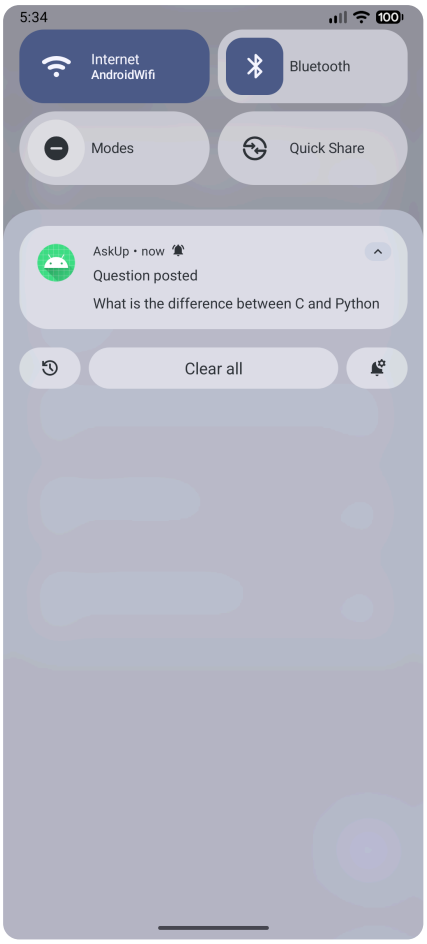
## 5.2 Notification System

The notification system informs users about events related to their questions. When a student posts a question, the system displays a notification confirming the submission.

Kotlin

```kotlin
val channel = NotificationChannel(
    CHANNEL_ID,
    CHANNEL_NAME,
    NotificationManager.IMPORTANCE_DEFAULT
)
```

The notification channel defines properties for all notifications from the application.




## 5.3 Dark Mode Implementation

Dark mode provides an alternative color scheme optimized for low light environments. The feature reduces eye strain when using the application in dark rooms. The implementation allows users to toggle between light and dark themes through the hamburger menu.

```kotlin
MaterialTheme(
    colorScheme = if (isDarkMode) darkColorScheme()
        else lightColorScheme()
) {
```

The `MaterialTheme` composable accepts a `colorScheme` parameter that determines the colors used throughout the interface. The application evaluates the dark mode preference when setting content and passes either `darkColorScheme()` or `lightColorScheme()` to `MaterialTheme`.

The hamburger menu displays the current theme status using both text labels and emoji. The dark mode switch shows the boolean state visually, while text below the switch reads either "Enabled" or "Disabled". Additional text at the bottom of the accessibility section shows "Dark 🌙" or "Light ☀️" to reinforce the current theme.

## 5.4 Lifecycle Management

Activity lifecycle callbacks are used to track when an activity becomes visible or hidden. The app overrides five main lifecycle methods in each activity: onStart, onResume, onPause, onStop, and onDestroy. Each method writes a log message to the system log when it is called, showing the transitions between lifecycle states and demonstrating understanding of how the activity lifecycle works.

```kotlin
override fun onPause() {
    super.onPause()
    println("StudentActivity: onPause")
}
```

The `onPause` callback indicates the activity is losing focus and becoming partially visible. This state occurs when another activity comes to the foreground without completely covering the current activity.

The `onResume` callback represents the state where an activity is fully visible and interactive. This state indicates the activity is in the foreground and can receive user input.

The `onStop` callback signals complete invisibility. The activity remains in memory but is no longer visible to users.

---

# Data Structures and Algorithms

## 6.1 Primary Data Structures

```kotlin
var questions by remember {
    mutableStateOf<List<Question>>(emptyList())
}
```

The `MutableList` type allows modification of the collection contents. The application uses mutable lists when building collections that will change over time, such as accumulating results from database queries. Once a collection is complete, it typically converts to an immutable List for passing to other functions or displaying in the interface.

## 6.2 Sorting and Filtering Logic

```kotlin
ORDER BY isHighlighted DESC, upvotes DESC, timestamp DESC
```

The first key, `isHighlighted` in descending order, partitions pinned questions from unpinned ones. Within each partition, the upvotes key sorts by popularity. The timestamp key provides a tiebreaker for questions with equal upvote counts.

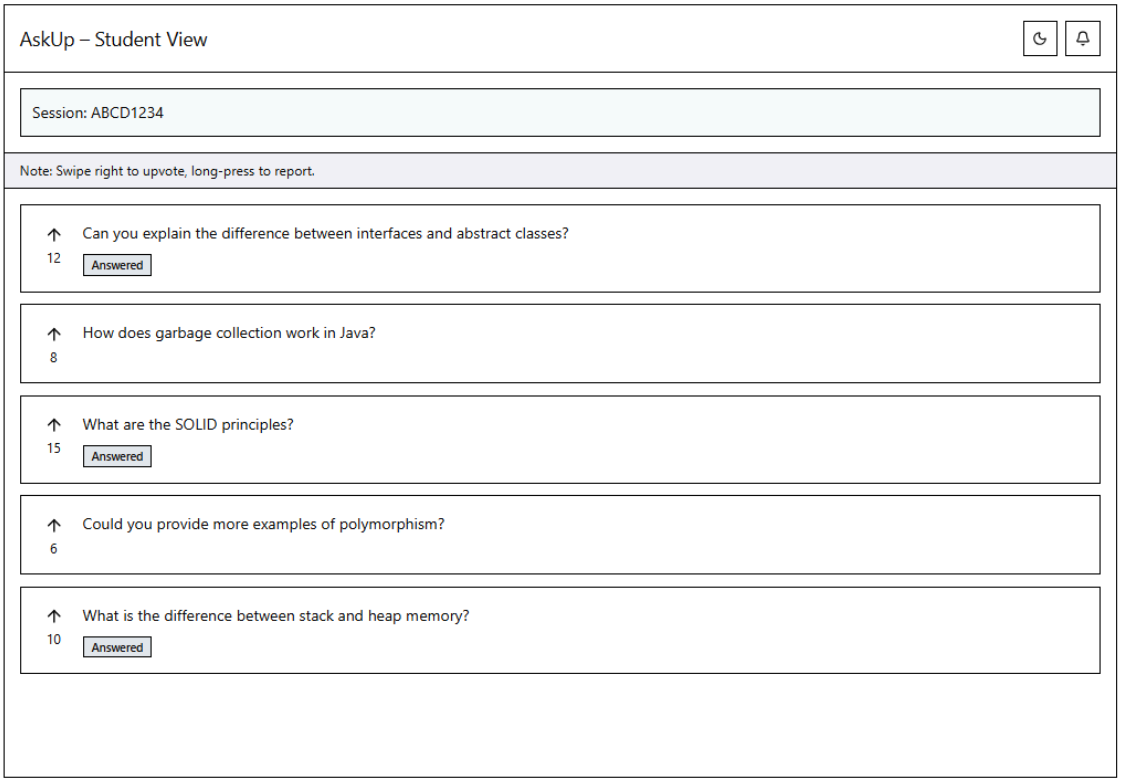
---

# Design Evolution from Wireframes

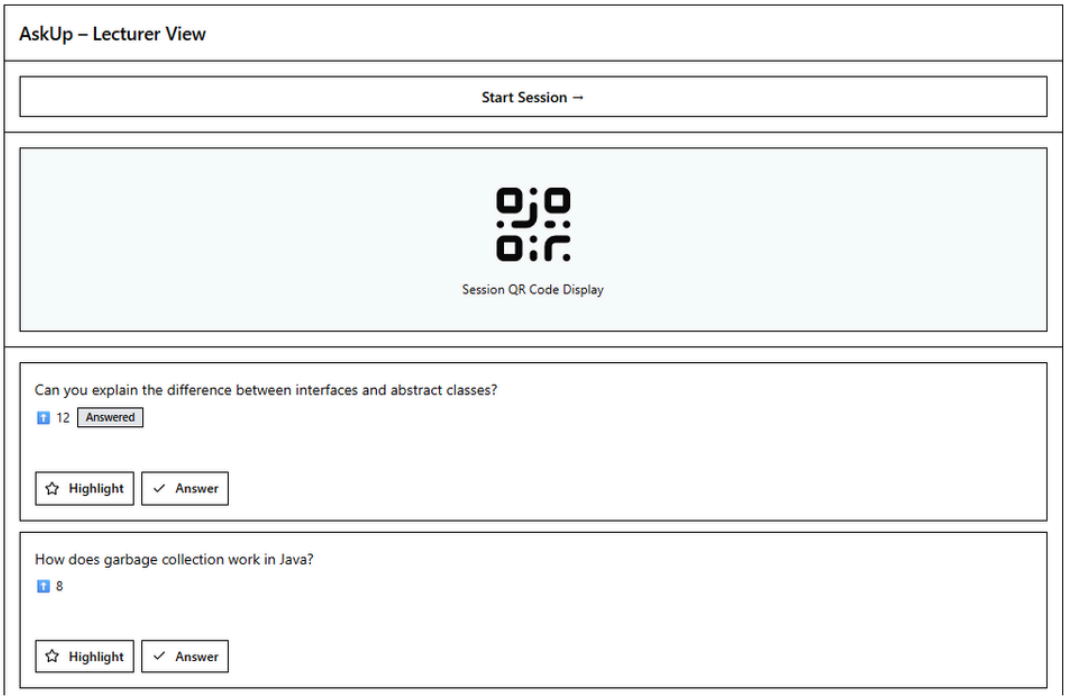
The implementation simplified the original wireframe concept of QR code session joining to a 5 digit numeric code system. This change avoided requiring additional libraries while maintaining the same functionality. Numeric codes can be communicated verbally during class and work reliably without camera dependencies. The student and lecturer views also converged to use consistent card layouts, with role differences expressed through available actions rather than completely different interfaces. This consistency emerged from recognizing that both roles interact with the same question data structure.

**AskUp Wireframe**

Student View



Lecturer View




---

# Conclusion

**AskUp** successfully addresses the challenge of encouraging classroom participation through a digital question and answer platform. The application meets all assignment requirements. Future enhancements could include real time synchronization across devices using cloud services and analytics showing participation patterns to help lecturers understand classroom engagement. Through this project, I learned a lot about Android development, including Jetpack Compose for reactive UIs, Room databases, activity lifecycles, and sensor integration for speech-to-text.

---